Nehru Institute of Technology

(Autonomous)

Approved by AICTE, New Delhi & Affiliated to Anna University, Chennai
Recognized by UGC with Section 2(f), Accredited by NAAC with A+
"Jawahar Gardens", Kaliyapuram, Coimbatore 641105

Non written Test II: Mini Project

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

MINI PROJECT REPORT

CLOUD INFRASTRUCTURE SLA MONITORING SYSTEM

VII SEMESTER

NAME	**BALAMURUGAN P G**
REG. NO.	721023104019
CLASS	4 CSE A
SUBJECT	CLOUD COMPUTING VIRTUALIZATION
SUBJECT CODE	U23CS702

Submitted as a mini project for

Cloud Computing Virtualization

Non written Test II: Mini Project

Name : BALAMURUGAN P G

Reg. No. : 721023104019

Class : 4 CSE A

Subject : U23CS702 - Cloud Computing Virtualization

Mark Evaluation Scheme

Component	Max. Marks	Marks Obtained
Problem identification and objectives	5	
Design and methodology	5	
Implementation	10	
Live demonstration and output	10	
Results and analysis	5	
Viva voce	5	
TOTAL	40	

Date :

Course Instructor

Dr S Pathur Nisha

Institution Vision

To be leading Institution in Academic excellence, Multidisciplinary Research, Innovation, Entrepreneurship and Industry relations in order to mould true citizens of the country.

Institution Mission

To create innovative and vibrant young leaders in the field of Engineering and Technology to grow India as knowledge power by strengthening the teaching-learning process.

To enhance employability, entrepreneurship and to improve the research competence to address societal needs.

To generate engineering graduates who use knowledge as a powerful tool to drive societal transformation and inculcate ethical and moral values.

Department Vision

To become a School of Computing excellence in research and innovation by imparting industry driven Teaching Learning Process.

Department Mission

To inculcate an innovator by adopting student-centric, activity and outcome-based teaching learning process in diversified areas of Computer Science and Engineering.

To achieve global standards with technical transformations in education and value-based living through a social and scientific approach.

To groom graduates with all-round leadership qualities and team spirit to meet the requirements of industry, business and society.

ACKNOWLEDGEMENT

I acknowledge Nehru Institute of Technology, Coimbatore, and the Department of Computer Science and Engineering for the academic setting in which this mini project was developed for U23CS702 Cloud Computing Virtualization.

I express my gratitude to the course faculty for the opportunity to study cloud service reliability through a practical implementation. Preparing this project helped me connect service level agreements, monitored performance indicators and cloud operations with an interactive software application.

I acknowledge the developers and maintainers of Python, Streamlit, pandas, NumPy and Plotly, whose open-source tools support the application. I also acknowledge the researchers and documentation authors cited in this report for the concepts used to understand cloud computing and service reliability.

I thank my family and friends for their encouragement during the preparation of this mini project and report.

BALAMURUGAN P G

721023104019

ABSTRACT

Cloud services must be evaluated against measurable reliability and performance commitments. Individual metric readings can show the current state of a service, but a reporting window is needed to identify repeated violations and understand compliance over time. This mini project presents a Cloud Infrastructure SLA Monitoring System that monitors uptime, response time, error rate and availability through an interactive dashboard. The core algorithm is rule-based threshold comparison. Each sampled observation is checked against a configurable upper or lower limit. Passing observations are counted to calculate per-metric compliance, and their four percentages are averaged to obtain a composite compliance score. Violations are recorded with timestamps, actual readings, targets, deviations and severity labels. Statistical summaries, rolling averages, an estimated availability error budget and rule-based observations support interpretation. The application uses Python, Streamlit, pandas, NumPy and Plotly. It provides a reproducible simulator, stress scenarios, adaptive sampling intervals, configurable date windows, CSV export and an optional Google Cloud Monitoring adapter. Visual outputs include metric sparklines, time-series comparisons, a compliance radar, a breach timeline, hourly heatmaps and a latency histogram. Light and pure-black dark themes support different presentation settings. Local validation consists of 22 automated checks covering calculations, controls, data handling and visual summaries. The included screenshots demonstrate simulated telemetry; authenticated cloud ingestion was not validated in this submission. The result is an explainable educational monitoring tool that detects observed threshold breaches without claiming predictive machine learning or automatic remediation.

TABLE OF CONTENTS

CHAPTER NO. / TITLE	PAGE NO.
1 Introduction	1
2 Literature Survey	2
2.1 Review of related work	2
2.2 Comparative analysis and project scope	3
3 Proposed System and Work	4
3.1 System architecture	4
3.2 Algorithm and methodology	5
3.3 Implementation and features	6
3.4 Validation and results	7
3.5 Output screenshots	8-10
4 Conclusion	11
Reference	12

LIST OF FIGURES

FIGURE	PAGE NO.
3.1 Dark theme dashboard overview	8
3.2 Visual analytics and incident pulse	9
3.3 Light theme dashboard overview	10

CHAPTER 1
INTRODUCTION

1.1 Background

Cloud computing provides shared computing resources that can be provisioned on demand. Virtual machines, containers and managed services allow applications to scale, but resource availability alone does not guarantee acceptable user experience. A service can remain reachable while its response time or error rate becomes unacceptable. The NIST definition establishes the cloud service context for this project [1].

A service level indicator is a measured property such as latency or availability. A service level objective specifies a desired value, while a service level agreement may attach contractual consequences to unmet commitments [2]. This project demonstrates the measurement and comparison part of that process; it does not calculate contractual penalties.

1.2 Problem statement

Checking separate charts manually makes it difficult to distinguish a temporary spike from repeated threshold failures. A healthy latest reading can also hide poor compliance earlier in the selected window. A compact application is therefore needed to evaluate multiple metrics consistently, retain violation evidence and explain the results in a form suitable for a live demonstration.

1.3 Objectives

Monitor four service indicators and compare every available sample with configurable targets.

Calculate per-metric and composite compliance without hiding earlier breaches behind an average reading.

Provide reproducible simulation and stronger stress scenarios for demonstrations.

Support investigation through interactive charts, an incident log, filtering and CSV export.

1.4 Scope and connection to CCV

The work concerns monitoring services hosted on cloud or virtualized infrastructure. The software runs locally and can optionally query Google Cloud Monitoring. It does not create virtual machines, manage hypervisors or automatically repair a failed service. Its focus is an explainable SLA evaluation workflow over sampled telemetry.

CHAPTER 2
LITERATURE SURVEY

2.1 Review of related work

Mell and Grance describe cloud computing through essential characteristics, service models and deployment models [1]. Their definition gives a foundation for understanding why shared resources require observable service behavior. The document does not provide an SLA dashboard algorithm, so this project adds a practical metric evaluation layer.

Jones, Wilkes and Murphy distinguish indicators, objectives and agreements, and discuss aggregation and error budgets [2]. These concepts guide the separation of latest readings from compliance over a window. The project uses a simple sample-based score, while clearly distinguishing that score from a contractual availability measurement.

Emeakaroha, Brandic, Maurer and Dustdar propose the LoM2HiS framework for mapping low-level resource metrics to high-level SLA parameters [3]. Their architecture connects monitoring, metric mapping and SLA management. The present mini project adopts the narrower idea of converting observations into explicit SLA evidence; it does not implement the paper's autonomic enforcement infrastructure or violation prediction.

Ewaschuk discusses monitoring distributed systems and the use of latency, traffic, errors and saturation as useful signals [4]. The selected dashboard includes latency, traffic and errors together with uptime and availability. Saturation measurements such as CPU or memory utilization are outside the implemented metric set.

Prometheus documentation demonstrates expression-based alert rules and configurable waiting periods before an alert fires [5]. This supports the practical value of transparent conditions. The mini project evaluates each sample directly and records incidents; it does not implement Alertmanager delivery or sustained-duration alert rules.

Google Cloud Monitoring provides metric collection, queries, dashboards and alerting services [6]. Its API is the optional telemetry source in this project. Streamlit and Plotly provide the application and visualization capabilities used to present the evaluated observations [7], [8].

2.2 Comparative analysis and project scope

Source / approach	Relevant contribution	Relationship to this project
NIST cloud definition [1]	Cloud characteristics and service models.	Establishes the cloud computing context.
Google SRE objectives [2]	SLI/SLO/SLA separation and error budgets.	Guides metric interpretation and budget estimates.
LoM2HiS framework [3]	Mapping monitored metrics to SLA parameters.	Motivates explainable evaluation; no autonomic enforcement is claimed.
Distributed systems monitoring [4]	Operational signals and monitoring principles.	Guides the choice of latency, errors and traffic views.
Prometheus rules [5]	Explicit expressions for alert conditions.	Supports deterministic rules; external alert delivery is not included.
Cloud Monitoring [6]	Managed metric collection and API access.	Supplies an optional read-only integration path.

2.3 Identified need

The reviewed work establishes strong monitoring principles, but a mini project also needs a small, reproducible environment that can be demonstrated without a production cloud account. The identified need is educational integration: collect or simulate samples, apply visible targets, show calculations and preserve investigation evidence in one application.

The proposed contribution is this integrated workflow, not a new learning algorithm or a replacement for production monitoring platforms. Reproducible simulation makes threshold changes and stress tests easy to explain, while the optional cloud adapter shows how the same evaluation pipeline can consume external telemetry.

CHAPTER 3
PROPOSED SYSTEM AND WORK

3.1 System architecture

The system separates acquisition, analysis and presentation. A shared tabular schema allows simulated and eligible cloud observations to pass through the same threshold evaluation functions. The dashboard presents the source label so that simulated values are not mistaken for measured production data.

STAGE	PROCESSING AND OUTPUT
1 Data source	Seeded simulator OR read-only Google Cloud Monitoring queries.
2 Sample preparation	Timestamped uptime, response time, error rate, availability and request counts.
3 SLA analyzer	Directional threshold checks, compliance percentages and a sorted violation log.
4 Supporting analysis	Availability loss estimate, rolling averages, quantiles and rule-based observations.
5 Dashboard and export	Overview, interactive visual analytics, incident filters, raw data and CSV export.

3.1.1 Data acquisition

The simulator uses a fixed random seed, probability distributions, time-of-day patterns and controlled violation injection. Stress mode increases anomaly frequency and severity. Changing evaluation thresholds does not regenerate the observations, allowing a fair comparison against the same snapshot.

The cloud adapter queries uptime checks and Cloud Run request and latency metrics, with an HTTPS load-balancer alternative when Cloud Run request data is absent. Requests are grouped by response code to derive a 5xx error percentage, and latency means are weighted by distribution counts. In this integration, availability reuses the uptime-check observations rather than acting as an independent measurement.

3.1.2 Failure handling

Missing credentials, unavailable streams or incomplete aligned coverage produce an explicit simulation fallback. Missing cloud values are not replaced with fixed healthy readings. This prevents incomplete observations from silently overstating reliability.

3.2 Algorithm and methodology

The primary algorithm is rule-based threshold comparison, implemented with vectorized pandas and NumPy operations. It is deterministic for a fixed input and target configuration and does not require a training dataset or a machine-learning model.

Metric	Passing condition	Default target
Uptime	Observed value is at least the target	99.9%
Response time	Observed value is at most the target	200 ms
Error rate	Observed value is at most the target	0.1%
Availability	Observed value is at least the target	99.95%

3.2.1 Processing steps

Step 1 Select the source, time window, sampling interval and four SLA targets.

Step 2 For each metric, compare every sample using the correct upper-bound or lower-bound condition.

Step 3 Divide passing samples by total samples and multiply by 100 to obtain metric compliance.

Step 4 Calculate the simple arithmetic mean of the four compliance percentages as the composite score.

Step 5 For failed samples, record time, metric, observed value, target, absolute deviation and severity.

Step 6 Sort violations newest first and prepare charts, summaries, diagnostic observations and exports.

3.2.2 Worked example and severity

If 95 of 100 response-time samples are within 200 ms, response-time compliance is 95%. Four compliance rates of 100%, 95%, 98% and 99% produce a composite score of 98%. The latest reading is evaluated separately, so it can be in target even when the window contains earlier failures.

The current severity rule uses absolute deviation in the metric's own unit: Warning below 0.05, Moderate from 0.05 to below 0.5, and Critical at or above 0.5. These buckets should be compared within a metric because milliseconds and percentage points have different scales. Scanning four metrics takes O(N) time for N samples; sorting V violations adds O(V log V).

3.3 Implementation and features

Module / technology	Responsibility
app.py / Streamlit	Views, controls, snapshot state, incident filters and downloads.
sla_simulator.py / NumPy	Reproducible telemetry and stress scenarios.
sla_analyzer.py / pandas	Compliance checks, violation log and rule-based observations.
gcp_metrics.py	Read-only Cloud Monitoring acquisition and explicit fallback.
dashboard_data.py	Date validation, estimated error budget and hourly breach aggregation.
dashboard_ui.py / Plotly	Sparklines, time series, radar, comparison panels and latency histogram.
dashboard_theme.py and CSS	Visible Light/Dark controls, preserved settings and responsive visuals.

3.3.1 Time windows and sampling

The user can select 24 hours, seven days, 30 days or custom dates covering up to 60 calendar days. The simulator uses five-minute intervals for windows up to 24 hours, ten minutes up to 72 hours, fifteen minutes up to seven days, thirty minutes up to 30 days and sixty minutes for longer windows. This controls the number of rendered points without claiming a measured rendering-time guarantee.

3.3.2 Visual investigation

The overview separates latest values from window compliance. Users can zoom time-series charts, compare all four metrics with independent units, view a 0-100% compliance radar and inspect a breach ribbon. Ribbon tiles summarize consecutive samples using the maximum simultaneous breach count. The heatmap aggregates failed-check percentages by date and hour.

The histogram shows the distribution of sampled interval latency values with median, P95 and SLA markers. These percentiles describe the supplied interval readings, not individual-request latency percentiles. The incident workspace supports metric, severity and text filters, while the data view exposes raw telemetry. Both views provide CSV downloads.

3.4 Validation and results

Local validation was performed using unit tests and Streamlit AppTest. The complete suite contained 22 checks and passed during report preparation. The checks establish local calculation and interaction behavior; they do not establish production throughput, detection accuracy against labeled outages or authenticated cloud connectivity.

Validation area	Examples of checks	Outcome
Sampling and dates	Exact five-minute spacing, incomplete dates, 60-day limit and future-time exclusion.	Passed
Compliance workflow	Stable telemetry during threshold changes, stress mode and longer windows.	Passed
Budget and gaps	Partial availability loss, zero budget and exclusion of missing intervals.	Passed
Incident investigation	Metric filtering, no-match state and visible fallback without a project ID.	Passed
Cloud data assembly	Weighted latency, request-derived errors and rejection of incomplete streams.	Passed
Appearance and visuals	Theme-state preservation, histogram totals, comparison values and radar scale.	Passed

3.4.1 Interpretation of the illustrated output

Figures 3.1-3.3 show one simulated 24-hour snapshot with 288 samples at five-minute spacing. The dashboard displays approximately 93.6% composite compliance and 74 metric violations, while all four latest readings are in target. This demonstrates why a latest-value display alone is insufficient. One interval can contain more than one metric violation, so 74 violations do not mean 74 distinct outages.

3.4.2 Limitations

The error budget estimates loss from sampled availability percentages over observed intervals; it is not measured outage duration. Diagnostic text is heuristic and cannot establish root cause. The application uses snapshots refreshed by the user and does not currently provide continuous background alert delivery, persistent incident storage, automatic remediation or predictive machine learning. Authenticated GCP ingestion requires separate validation with a configured cloud project.

3.5 Output screenshots

3.5.1 Dashboard overview in dark theme

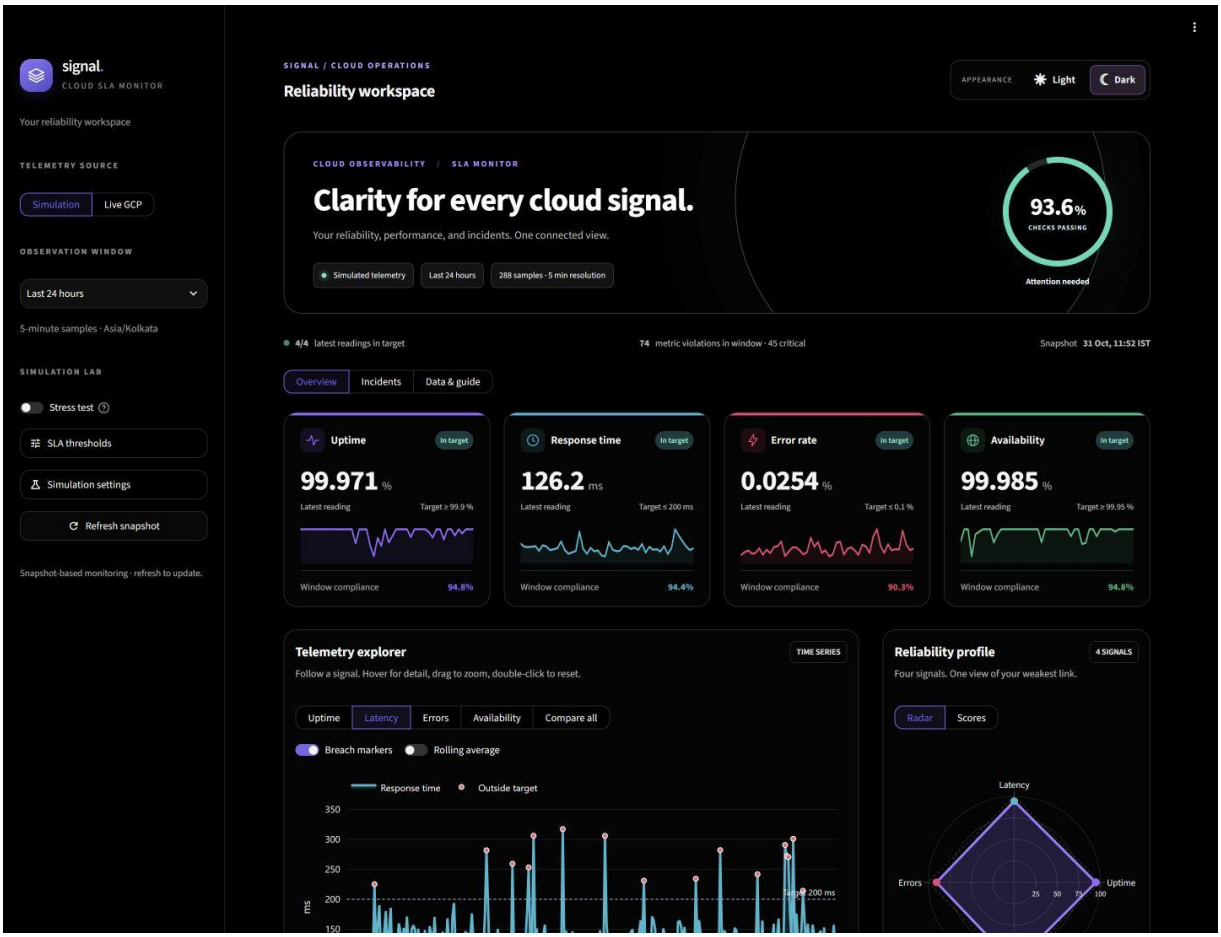


Figure 3.1 Dark theme dashboard overview using simulated telemetry

The overview displays the selected source, monitoring window, sample count and composite score. Each metric card shows the latest value, target and a miniature trend, followed by window compliance. The screenshot also shows the visible appearance controls and the beginning of the telemetry explorer and radar panel.

In this snapshot, current readings meet their targets even though earlier samples caused violations. The source is explicitly identified as simulated telemetry; the screen is an application output rather than evidence of a live cloud deployment.

3.5.2 Visual analytics and incident timeline

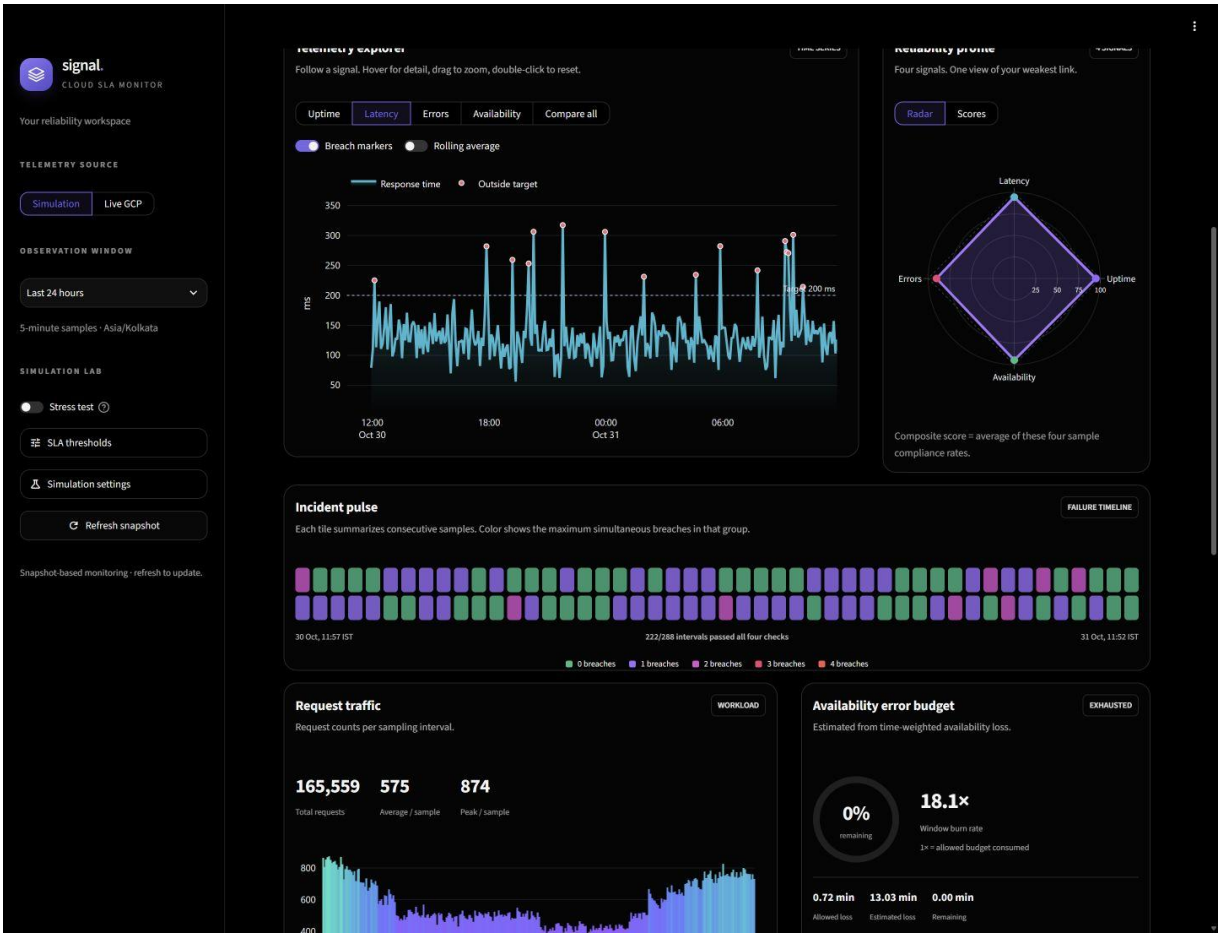


Figure 3.2 Time-series analysis radar profile and incident pulse

The time-series chart places observations against a target boundary so that individual breaches can be investigated. The radar compares the four compliance rates on a common 0-100 scale. The incident pulse provides a separate visual view of concurrent failures; each tile represents a group of samples and reports its maximum breach count on hover or keyboard focus.

The lower panels show request traffic and the estimated availability error budget. Together, these views support investigation of operational patterns. Visual correlation is useful for choosing what to investigate next, but does not prove that traffic caused an observed failure.

3.5.3 Dashboard overview in light theme

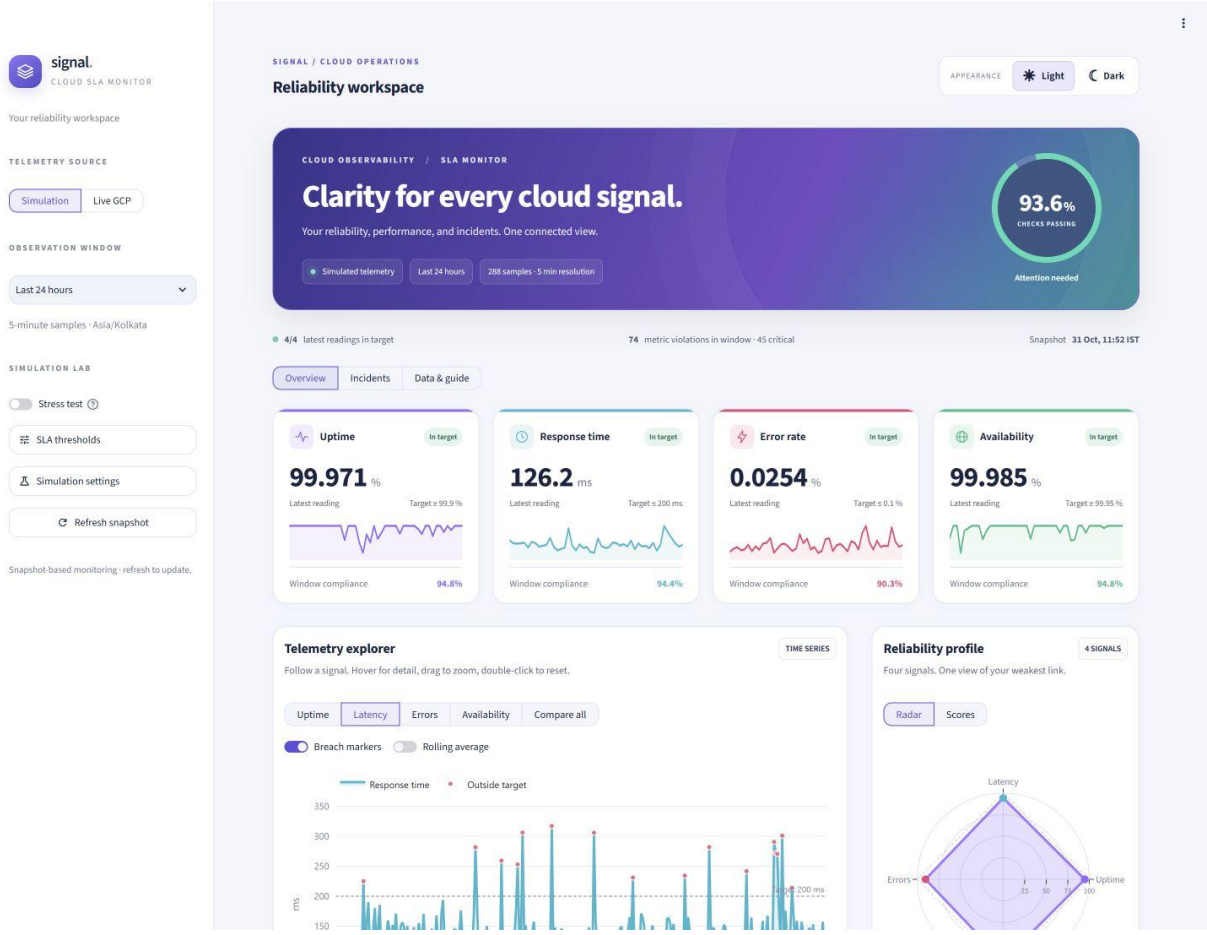


Figure 3.3 Light theme dashboard overview using the same snapshot

The light theme provides an alternative presentation using the same monitoring controls and visual structure. The appearance links switch the native theme so that controls, charts and tabular views remain coordinated. Relevant settings and the snapshot are retained during appearance navigation.

The figures demonstrate output presentation and local interaction. They should be read with the validation results and limitations in Section 3.4, rather than treated as a benchmark or an authenticated production monitoring result.

CHAPTER 4
CONCLUSION

The Cloud Infrastructure SLA Monitoring System demonstrates an end-to-end workflow for evaluating service indicators against configured targets. It combines reproducible telemetry, directional threshold checks, sample-based compliance, a violation log and interactive investigation views in a single application.

The implemented algorithm is transparent: an observation either satisfies its configured condition or is recorded as a violation. The composite score summarizes the four compliance rates, while separate latest-value and window views prevent current healthy readings from hiding earlier failures. The incident workspace and CSV exports preserve evidence for further analysis.

The project also demonstrates practical cloud monitoring considerations. Missing external telemetry is handled explicitly, cloud request data is aligned by timestamp, and the availability budget excludes gaps rather than inventing observations. The interface provides both a pure-black dark theme and a readable light theme, together with time-series, radar, timeline, heatmap and distribution views.

4.1 Achieved outcome

All 22 local automated checks passed. The included outputs show that the application can present simulated service degradation, distinguish latest readings from historical compliance and support filtered investigation. The work meets its educational objective of explaining SLA monitoring through an inspectable implementation and a reproducible demonstration.

4.2 Future work

Future extensions could include independently measured availability, metric-specific severity scales, persistent incident storage, scheduled collection and notification delivery. Authenticated cloud testing should be completed before production use. Forecasting could be evaluated as a separate enhancement using historical labeled data, with its prediction quality measured independently from the current rule-based detector.

REFERENCE

[1] P. Mell and T. Grance, The NIST Definition of Cloud Computing, NIST Special Publication 800-145, September 2011. DOI: 10.6028/NIST.SP.800-145. https://csrc.nist.gov/pubs/sp/800/145/final

[2] C. Jones, J. Wilkes and N. R. Murphy, with C. Smith, "Service Level Objectives," in Site Reliability Engineering, Google, 2016. https://sre.google/sre-book/service-level-objectives/

[3] V. C. Emeakaroha, I. Brandic, M. Maurer and S. Dustdar, "Low level Metrics to High level SLAs - LoM2HiS framework: Bridging the gap between monitored metrics and SLA parameters in cloud environments," International Conference on High Performance Computing and Simulation, pp. 48-54, 2010. DOI: 10.1109/HPCS.2010.5547150.

[4] R. Ewaschuk, "Monitoring Distributed Systems," in Site Reliability Engineering, edited by B. Beyer et al., Google, 2016. https://sre.google/sre-book/monitoring-distributed-systems/

[5] Prometheus Authors, "Alerting rules," Prometheus documentation. https://prometheus.io/docs/prometheus/latest/configuration/alerting_rules/

[6] Google Cloud, "Cloud Monitoring overview," Google Cloud documentation. https://docs.cloud.google.com/monitoring/docs/monitoring-overview

[7] Streamlit, "Streamlit documentation." https://docs.streamlit.io/

[8] Plotly Technologies Inc., "Python graphing library documentation." https://plotly.com/python/

Online documentation accessed on 31 October 2026.